

InfoTrack

Sistema de Seguimiento Académico para Estudiantes de Informática

Stack: React · HTML/CSS · Node.js · Apollo Server · GraphQL · MongoDB + Mongoose
Carreras cubiertas: Lic. Informática · Tec. Web · Tec. Redes · Analista de Sistemas
Versión 1.0 — Documentación completa + MVP — 2025

ENTREGA ACADÉMICA — USO INTERNO

1. Visión del producto

1.1 Qué es InfoTrack

InfoTrack es una plataforma web diseñada exclusivamente para estudiantes del área de Informática de la universidad. Centraliza en un solo lugar todo lo que un alumno necesita para gestionar su carrera: el seguimiento de materias aprobadas y pendientes, los profesores del cuatrimestre en curso, un calendario personal de tareas y evaluaciones, recomendaciones entre pares sobre docentes, recursos compartidos por materia, y una calculadora de promedio ponderado.

El sistema está construido sobre una API GraphQL, lo que permite a cada vista del cliente pedir exactamente los datos que necesita, sin over-fetching ni under-fetching, y facilita la escalabilidad a nuevas funcionalidades sin romper los clientes existentes.

1.2 Problema que resuelve

- Los alumnos no tienen una vista clara de cuánto les falta para recibirse.
- La información de profesores y materias del cuatrimestre está dispersa o es difícil de encontrar.
- No hay un espacio comunitario donde los propios alumnos compartan experiencias sobre docentes.
- El seguimiento de tareas y fechas de parciales se hace en herramientas genéricas no pensadas para el contexto universitario.
- Calcular el promedio ponderado con y sin aplazos requiere hacerlo a mano.

1.3 Carreras cubiertas

Carrera	Tipo · Duración	Descripción
Licenciatura en Informática	Grado · 5 años	Formación completa en CS, software, algoritmos y gestión de sistemas.
Tecnicatura en Tecnologías Web	Tec. · 3 años	Desarrollo web frontend, backend, UX y aplicaciones móviles.
Tecnicatura en Redes Informáticas	Tec. · 3 años	Infraestructura, redes, seguridad y administración de sistemas.
Analista de Sistemas	Analista · 3 años	Análisis, diseño, gestión y documentación de sistemas de información.

2. Roles y módulos del sistema

2.1 Roles

Rol	Descripción	Acceso
Estudiante	Usuario principal. Se registra con su carrera y legajo.	Dashboard, materias, cuatrimestre, calendario, recomendaciones, recursos, calculadora, perfil.
Admin	Personal de la facultad. Gestiona los datos base.	Todo lo del estudiante más: ABM de carreras, materias, profesores, usuarios y cuatrimestre activo.

2.2 Módulos del sistema

Módulo	Funcionalidad	Rol	MVP
Autenticación	Registro (carrera + legajo), login con JWT, perfil personal.	Ambos	Sí
Dashboard de progreso	% de carrera completado, aprobadas, en curso, promedio ponderado, materias restantes.	Estudiante	Sí
Plan de estudios	Vista de todas las materias de la carrera por año, con estado individual.	Ambos	Sí
Cuatrimestre en curso	Materias activas del cuatrimestre con sus profesores y horarios.	Ambos	Sí
Calendario de tareas	CRUD de tareas con fecha, tipo, prioridad y materia. Marcar como completadas.	Estudiante	Sí
Recomendaciones	Reseñas anónimas de profesores con rating y comentario. Botón 'útil'.	Estudiante	Sí
Recursos por materia	Compartir y ver links (apuntes, videos, ejercicios) por materia. Likes.	Estudiante	Sí
Calculadora de promedio	Calcula promedio ponderado con y sin aplazos según reglas de la carrera.	Estudiante	Sí
Panel de admin	ABM de carreras, materias, profesores, usuarios y cuatrimestre activo.	Admin	Sí
Mapa visual de carrera	Diagrama de materias y correlatividades, coloreado por estado.	Estudiante	v2
Simulador de cursada	Simular qué materias cursar sin afectar datos reales.	Estudiante	v2
Alertas de regularidad	Aviso cuando una regularidad está próxima a vencer.	Estudiante	v2

Módulo	Funcionalidad	Rol	MVP
Estadísticas personales	Gráficos de evolución del promedio y distribución por área.	Estudiante	v2

3. Stack tecnológico

3.1 Resumen del stack

Capa	Tecnología	Versión	Rol en el proyecto
Frontend — UI	React	18.x	SPA con componentes reutilizables y gestión de estado.
Frontend — Estilos	HTML + CSS Modules	—	Markup semántico y estilos con scope local por componente.
Frontend — Routing	React Router	v6	Navegación SPA con rutas protegidas por rol.
Frontend — GraphQL	Apollo Client	3.x	Queries, mutations, caché automático y estado del servidor.
Frontend — Gráficos	Recharts	2.x	Barras de progreso, gráfico de promedio y distribución.
Backend — Runtime	Node.js	20.x LTS	Entorno de ejecución del servidor.
Backend — API	Apollo Server	4.x	Servidor GraphQL standalone con Express como HTTP layer.
Backend — HTTP	Express	4.x	Middleware de autenticación, CORS y contexto de la request.
Backend — Schema	GraphQL SDL	—	Definición de tipos, queries, mutations y subscriptions.
Base de datos	MongoDB	7.x	Base documental NoSQL para datos flexibles.
ODM	Mongoose	8.x	Esquemas, validaciones, índices y hooks del modelo.
Autenticación	JWT + bcrypt	—	Tokens stateless enviados en el header de cada request GraphQL.
Configuración	dotenv	—	Variables sensibles en .env, nunca en el código fuente.

3.2 Por qué GraphQL en lugar de REST

- Una sola URL de endpoint (/graphql) para todas las operaciones del sistema.
- El cliente pide exactamente los campos que necesita: el Dashboard solo pide progress summary, no todos los enrollments.
- Apollo Client cachea los resultados automáticamente: navegar entre páginas no dispara requests repetidos.
- Introspección: el schema es autodocumentado y explorable con GraphQL Playground o Apollo Sandbox.

- Agregar nuevas funcionalidades (nuevos tipos y resolvers) no rompe los clientes existentes.
- Los tipos del schema GraphQL sirven como contrato entre frontend y backend, reduciendo errores de integración.

3.3 Arquitectura general

El flujo de una operación es: React (Apollo Client) → HTTP POST a /graphql → Express middleware (verifica JWT) → Apollo Server (valida el query contra el schema) → Resolver (lógica de negocio + consulta a MongoDB vía Mongoose) → respuesta JSON al cliente → Apollo Client actualiza el caché.

3.4 Estructura del proyecto

Backend — server/

```
server/
  src/
    config/
      db.js      → Conexión a MongoDB (mongoose.connect)
      env.js     → Variables de entorno tipadas y validadas
    models/      → Esquemas Mongoose (uno por colección)
      User.js    → Usuario: estudiante o admin
      Career.js  → Carrera universitaria
      Subject.js → Materia del plan de estudios
      Enrollment.js → Inscripción de un alumno a una materia
      Professor.js → Profesor
      CurrentSemester.js → Materias del cuatrimestre activo
      Task.js    → Tarea del calendario personal
      Review.js  → Reseña de un profesor
      Resource.js → Recurso compartido por materia
    schema/
      typeDefs/ → SDL por módulo: auth.graphql, user.graphql,
                  career.graphql, subject.graphql, enrollment.graphql,
                  professor.graphql, task.graphql, review.graphql,
resource.graphql
  index.js      → Merge de todos los typeDefs + resolvers
  resolvers/    → Un archivo por módulo de negocio
    authResolvers.js
    careerResolvers.js
    subjectResolvers.js
    enrollmentResolvers.js
    professorResolvers.js
    taskResolvers.js
    reviewResolvers.js
    resourceResolvers.js
  middleware/
    auth.js     → Extrae y verifica el JWT de req.headers.authorization
  utils/
    generateToken.js → Firma el JWT con el payload del usuario
    calcProgress.js  → Lógica de cálculo de % de carrera y promedio
    dataLoaders.js   → DataLoaders para evitar el problema N+1
```

```
index.js      → Entry point: ApolloServer + Express + context
.env
package.json
```

Frontend — client/

```
client/
src/
  apollo/
    client.js      → Configuración de ApolloClient con authLink + httpLink
  graphql/
    queries.js     → Todas las queries GraphQL (gql`...`)
    mutations.js   → Todas las mutations GraphQL
    fragments.js   → Fragments reutilizables
  context/
    AuthContext.jsx → Token JWT, usuario activo, login/logout
  components/
    → Componentes reutilizables
    ProgressBar/   → Barra de avance de la carrera
    SubjectCard/   → Card de materia con estado coloreado
    TaskCard/      → Card de tarea con checkbox y prioridad
    ReviewCard/    → Card de reseña con stars y útil
    ResourceCard/  → Card de recurso con tipo y like
    StatCard/      → Tarjeta de métrica (aprobadas, promedio, etc.)
    Modal/         → Wrapper reutilizable de modal
  pages/
    → Una carpeta por página
    LoginPage/     → Login y registro
    DashboardPage/ → Dashboard principal
    SubjectsPage/  → Plan de estudios
    SemesterPage/  → Cuatrimestre en curso
    CalendarPage/  → Tareas y calendario
    ReviewsPage/   → Recomendaciones de profesores
    ResourcesPage/ → Recursos por materia
    CalculatorPage/ → Calculadora de promedio
    ProfilePage/   → Perfil del estudiante
    AdminPage/     → Panel de administración
  routes/
    PrivateRoute.jsx → Redirige al login si no hay token
    AdminRoute.jsx   → Redirige al dashboard si no es admin
  utils/
    formatDate.js   → Formatea fechas en español
    subjectStatus.js → Retorna color/label según EnrollmentStatus
  App.jsx           → React Router con todas las rutas
  main.jsx          → Entry point: ApolloProvider + AuthProvider
index.html          → HTML base de Vite
package.json
```

4. Base de datos — Modelos MongoDB + Mongoose

El diseño de la base de datos sigue el patrón de referencias entre documentos (similar a relaciones) donde la consistencia de datos lo justifica, y el patrón de documentos embebidos donde la unidad de lectura es siempre la misma. Se detallan todos los campos, tipos, validaciones, índices y relaciones.

4.1 Career — Carrera universitaria

Contiene los datos de cada carrera. Los administradores la crean y mantienen. No se elimina si tiene alumnos activos (validación en resolver).

Campo	Tipo	Validación / Default	Descripción
_id	ObjectId	Auto	ID único generado por MongoDB.
name	String	required, trim	Nombre completo de la carrera.
shortName	String	required, maxLen:20	Nombre corto para mostrar en UI (ej: 'Lic. Informática').
type	String (enum)	required	'licenciatura' 'tecnicatura' 'analista'
durationYears	Number	required, min:1, max:6	Duración en años del plan de estudios.
description	String	optional	Descripción breve de la carrera para mostrar al registrarse.
totalCredits	Number	optional	Total de créditos/horas del plan (si aplica en la institución).
isActive	Boolean	default: true	Si la carrera está vigente. Las inactivas no aparecen en el registro.
createdAt / updatedAt	Date	timestamps: true	Gestionados automáticamente por Mongoose.

Índices: { name: 1 } único.

4.2 Subject — Materia del plan de estudios

Cada materia pertenece a una carrera y a un año/cuatrimestre del plan. Las correlatividades son referencias a otras materias de la misma colección. Un admin las carga y mantiene. No se elimina si tiene enrollments activos.

Campo	Tipo	Validación / Default	Descripción
_id	ObjectId	Auto	ID único.

Campo	Tipo	Validación / Default	Descripción
name	String	required, trim	Nombre oficial de la materia según el plan de estudios.
code	String	required, unique	Código único (ej: 'INF-205'). Usado para referenciar la materia.
career	ObjectId → Career	required, ref:'Career'	Carrera a la que pertenece esta materia.
year	Number	required, min:1, max:6	Año del plan en que se dicta esta materia.
semester	Number (enum)	required	1 o 2. Cuatrimestre dentro del año en que se dicta.
credits	Number	optional, min:0	Créditos o carga horaria de la materia.
area	String	optional	Área temática: 'Programación', 'Matemática', 'Redes', etc.
prerequisites	[ObjectId → Subject]	default: []	Correlativas requeridas. Referencia a otras materias del plan.
isElective	Boolean	default: false	Indica si es materia optativa o del plan obligatorio.
isActive	Boolean	default: true	Si la materia sigue vigente en el plan actual.
createdAt / updatedAt	Date	timestamps: true	Gestionados por Mongoose.

Índices: { code: 1 } único. { career: 1, year: 1, semester: 1 } compuesto para queries del plan de estudios.

4.3 User — Estudiante o administrador

Modelo central de autenticación. El password nunca se devuelve en ninguna query GraphQL (se excluye en el resolver). El campo enrollments se obtiene siempre mediante el resolver de Enrollment, no se guarda como array en User para evitar documentos excesivamente grandes.

Campo	Tipo	Validación / Default	Descripción
_id	ObjectId	Auto	ID único.
name	String	required, trim, minLen:2	Nombre completo del usuario.
email	String	required, unique, lowercase	Email de acceso. Debe ser único en toda la colección.
password	String	required, minLen:6	Hash bcrypt (saltRounds:12). Excluido de todas las queries.

Campo	Tipo	Validación / Default	Descripción
legajo	String	required, unique	Número de legajo universitario. Único por institución.
role	String (enum)	default:'student'	'student' 'admin'
career	ObjectId → Career	required si role=student	Carrera del estudiante. Null para admins.
startYear	Number	required si role=student	Año en que inició la carrera (ej: 2022).
avatar	String	optional	URL de imagen de perfil.
isActive	Boolean	default: true	Soft delete: los usuarios inactivos no pueden loguearse.
lastLogin	Date	optional	Fecha del último login exitoso.
createdAt / updatedAt	Date	timestamps: true	Gestionados por Mongoose.

Índices: { email: 1 } único. { legajo: 1 } único. { career: 1 } para listar alumnos por carrera.

Hook pre-save: antes de guardar, si el password fue modificado, se hashea con `bcrypt.hash(password, 12)`.

4.4 Enrollment — Inscripción de un alumno a una materia

Es el modelo más consultado del sistema. Representa la relación entre un estudiante y una materia, con su estado actual, nota y metadatos. Cada estudiante tiene como máximo un Enrollment por materia (índice compuesto único). A partir de los Enrollments se calculan el porcentaje de avance, el promedio y el estado de cada materia en el plan.

Campo	Tipo	Validación / Default	Descripción
_id	ObjectId	Auto	ID único.
student	ObjectId → User	required, ref:'User'	Estudiante dueño de esta inscripción.
subject	ObjectId → Subject	required, ref:'Subject'	Materia a la que corresponde.
status	String (enum)	required	'pending' 'enrolled' 'passed' 'failed' 'free'
grade	Number	min:1, max:10	Nota numérica. Solo presente si status es 'passed' o 'failed'.
approvedDate	Date	optional	Fecha exacta de aprobación del final. Solo si status='passed'.

Campo	Tipo	Validación / Default	Descripción
regularityExpires	Date	optional	Fecha en que vence la regularidad. Se calcula como approvedDate + 2 años.
enrolledSemester	String	required	Cuatrimestre en que se cursó (ej: '2025-1'). Formato YYYY-N.
attempts	Number	default:1, min:1	Cantidad de veces que rindió el final. Incrementa en cada intento.
notes	String	optional, maxLen:500	Notas personales del alumno sobre la materia.
createdAt / updatedAt	Date	timestamps: true	Gestionados por Mongoose.

Índices: { student: 1, subject: 1 } único — evita doble registro. { student: 1, status: 1 } para filtrar por estado. { student: 1, enrolledSemester: 1 } para agrupar por cuatrimestre.

Nota de diseño: el status 'free' (libre) indica que el alumno perdió la regularidad. El status 'pending' indica que todavía no la cursó. 'enrolled' es cursando actualmente.

4.5 Professor — Profesor

Creado y mantenido por el admin. Se relaciona con las materias que dicta. El campo avgRating y reviewCount se actualizan mediante un hook post-save en Review (desnormalización controlada para evitar agregaciones costosas en cada query).

Campo	Tipo	Validación / Default	Descripción
_id	ObjectId	Auto	ID único.
name	String	required, trim	Nombre completo del profesor.
email	String	optional, unique sparse	Email institucional. Sparse permite múltiples documentos sin email.
department	String	optional	Departamento o cátedra al que pertenece.
subjects	[ObjectId → Subject]	default:[], ref:'Subject'	Materias que dicta actualmente.
avgRating	Number	default:0, min:0, max:5	Promedio de ratings desnormalizado. Se recalcula al guardar/borrar una Review.
reviewCount	Number	default:0	Cantidad de reseñas recibidas. Desnormalizado para mostrar sin aggregation.
isActive	Boolean	default:true	Los profesores inactivos no aparecen en el cuatrimestre activo.
createdAt / updatedAt	Date	timestamps: true	Gestionados por Mongoose.

Índices: { name: 'text' } para búsqueda full-text por nombre.

4.6 CurrentSemester — Cuatrimestre en curso

Almacena qué materias se están dictando en el cuatrimestre activo, con qué profesores y en qué horarios. Solo existe un documento activo a la vez (isActive: true). Los anteriores se desactivan cuando se carga el nuevo cuatrimestre.

Campo	Tipo	Validación / Default	Descripción
_id	ObjectId	Auto	ID único.
label	String	required	Nombre del cuatrimestre (ej: '1° Cuatrimestre 2025').
code	String	required, unique	Código único del cuatrimestre (ej: '2025-1').
startDate	Date	required	Fecha de inicio del cuatrimestre.
endDate	Date	required	Fecha de fin del cuatrimestre.
isActive	Boolean	default:false	Solo un documento puede tener isActive:true a la vez.
offerings	[SemesterOffering]	required	Array embebido de materias dictadas (ver subdocumento abajo).
createdAt / updatedAt	Date	timestamps: true	Gestionados por Mongoose.

Subdocumento SemesterOffering (embebido en el array offerings):

- subject: ObjectId → Subject (required) — materia ofrecida
- professors: [ObjectId → Professor] (required) — docentes a cargo
- schedule: String (optional) — descripción del horario (ej: 'Lun y Mié 18:00-21:00')
- classroom: String (optional) — aula o modalidad (ej: 'Aula 12' o 'Virtual')
- career: ObjectId → Career (required) — para filtrar por carrera del alumno

Índices: { isActive: 1 } para encontrar el cuatrimestre activo. { 'offerings.subject': 1 } para buscar en qué cuatrimestre se ofrece una materia.

4.7 Task — Tarea del calendario personal

Privada por alumno. Nunca se comparte con otros usuarios. Los datos son del alumno, para el alumno.

Campo	Tipo	Validación / Default	Descripción
_id	ObjectId	Auto	ID único.

Campo	Tipo	Validación / Default	Descripción
student	ObjectId → User	required, ref:'User'	Estudiante dueño de la tarea.
title	String	required, trim, maxLen:200	Título descriptivo de la tarea.
description	String	optional, maxLen:1000	Detalle o apuntes sobre la tarea.
dueDate	Date	required	Fecha y hora límite de la tarea.
type	String (enum)	required	'tarea' 'parcial' 'final' 'proyecto' 'otro'
priority	String (enum)	default:'media'	'alta' 'media' 'baja'
completed	Boolean	default:false	Estado de la tarea.
completedAt	Date	optional	Timestamp de cuándo se marcó como completada.
subject	ObjectId → Subject	optional, ref:'Subject'	Materia relacionada (para filtrar y mostrar en la card).
createdAt / updatedAt	Date	timestamps: true	Gestionados por Mongoose.

Índices: { student: 1, dueDate: 1 } compuesto para traer las tareas ordenadas por fecha. { student: 1, completed: 1 } para filtrar pendientes vs. completadas.

4.8 Review — Reseña de un profesor

Un estudiante puede escribir una sola reseña por combinación (profesor + materia). Después de guardar o borrar una Review, un hook post-save recalcula avgRating y reviewCount en el documento Professor correspondiente para mantenerlos sincronizados sin aggregations en cada request.

Campo	Tipo	Validación / Default	Descripción
_id	ObjectId	Auto	ID único.
professor	ObjectId → Professor	required, ref:'Professor'	Profesor reseñado.
subject	ObjectId → Subject	required, ref:'Subject'	Materia en que el alumno tuvo a este profesor.
student	ObjectId → User	required, ref:'User'	Autor de la reseña.
rating	Number	required, min:1, max:5	Calificación del 1 al 5.

Campo	Tipo	Validación / Default	Descripción
comment	String	required, minLen:20, maxLen:800	Texto de la reseña. Mínimo 20 caracteres para evitar ruido.
isAnonymous	Boolean	default:true	Si true, no se expone el nombre del autor en la UI.
helpfulCount	Number	default:0	Votos 'me fue útil' de otros alumnos.
helpfulVoters	[ObjectId → User]	default:[]	IDs de usuarios que votaron útil. Evita doble voto.
isVisible	Boolean	default:true	Soft delete: admin puede ocultar reseñas inapropiadas.
createdAt / updatedAt	Date	timestamps: true	Gestionados por Mongoose.

Índices: { professor: 1, subject: 1, student: 1 } único — un alumno, una reseña por materia+profesor. { professor: 1, rating: -1 } para listar por mejor calificados. { isVisible: 1 } para filtrar las visibles.

4.9 Resource — Recurso compartido por materia

Contenido comunitario. Cualquier estudiante puede compartir un link. Otros alumnos pueden votar que les fue útil. Un mismo estudiante no puede votar dos veces (helpfulVoters evita duplicados).

Campo	Tipo	Validación / Default	Descripción
_id	ObjectId	Auto	ID único.
subject	ObjectId → Subject	required, ref:'Subject'	Materia a la que corresponde el recurso.
student	ObjectId → User	required, ref:'User'	Estudiante que lo compartió.
title	String	required, trim, maxLen:200	Título descriptivo del recurso.
url	String	required, validate:isURL	URL del recurso. Validada como URL válida por Mongoose.
type	String (enum)	required	'apunte' 'video' 'ejercicio' 'libro' 'otro'
description	String	optional, maxLen:400	Breve descripción del contenido.
likeCount	Number	default:0	Cantidad de 'me sirvió'. Desnormalizado para ordenar sin aggregation.
likeVoters	[ObjectId → User]	default:[]	IDs de usuarios que dieron like. Evita doble voto.

Campo	Tipo	Validación / Default	Descripción
isVisible	Boolean	default:true	Soft delete: admin puede ocultar recursos inapropiados.
createdAt / updatedAt	Date	timestamps: true	Gestionados por Mongoose.

Índices: { subject: 1, likeCount: -1 } para listar recursos de una materia ordenados por más útiles. { student: 1 } para ver los recursos que compartió un alumno.

5. Schema GraphQL — Tipos y operaciones

5.1 Tipos escalares y enums

```
enum UserRole { student admin }
enum EnrollmentStatus { pending enrolled passed failed free }
enum TaskType { tarea parcial final proyecto otro }
enum TaskPriority { alta media baja }
enum ResourceType { apunte video ejercicio libro otro }
enum CareerType { licenciatura tecnicatura analista }
enum SubjectSemester { ONE TWO }
```

5.2 Tipos de objeto principales

```
type Career {
  _id: ID!
  name: String!
  shortName: String!
  type: CareerType!
  durationYears: Int!
  description: String
  totalCredits: Int
  isActive: Boolean!
  subjects: [Subject!]!      # resolver: busca por career._id
  totalSubjects: Int!       # resolver: cuenta subjects activas
}
```

```
type Subject {
  _id: ID!
  name: String!
  code: String!
  career: Career!
  year: Int!
  semester: SubjectSemester!
  credits: Int
  area: String
  prerequisites: [Subject!]!
  isElective: Boolean!
  isActive: Boolean!
}
```

```
type User {
  _id: ID!
  name: String!
  email: String!
  # password: nunca expuesto
  legajo: String!
  role: UserRole!
  career: Career
  startYear: Int
  avatar: String
  isActive: Boolean!
  lastLogin: String
}
```



```
  enrollments: [Enrollment!]! # resolver: filtra por student._id
  progress: ProgressSummary! # resolver: calcula en base a enrollments
}
```

```
type ProgressSummary {
  totalSubjects: Int! # total de materias del plan de la carrera
  passed: Int! # aprobadas (status: passed)
  failed: Int! # desaprobadas (status: failed)
  enrolled: Int! # cursando (status: enrolled)
  pending: Int! # sin cursar (status: pending)
  free: Int! # libre (status: free)
  percentage: Float! # passed / totalSubjects * 100
  average: Float! # promedio de grades donde status=passed
  weightedAverage: Float! # promedio ponderado por créditos
}
```

```
type Enrollment {
  _id: ID!
  student: User!
  subject: Subject!
  status: EnrollmentStatus!
  grade: Float
  approvedDate: String
  regularityExpires: String
  enrolledSemester: String!
  attempts: Int!
  notes: String
  updatedAt: String!
}
```

```
type Professor {
  _id: ID!
  name: String!
  email: String
  department: String
  subjects: [Subject!]!
  avgRating: Float!
  reviewCount: Int!
  isActive: Boolean!
  reviews: [Review!]! # resolver: busca por professor._id, isVisible:true
}
```

```
type SemesterOffering {
  subject: Subject!
  professors: [Professor!]!
  schedule: String
  classroom: String
  career: Career!
}
```

```
type CurrentSemester {
  _id: ID!
  label: String!
  code: String!
}
```

```
    startDate: String!
    endDate: String!
    isActive: Boolean!
    offerings: [SemesterOffering!]!
  }
}
```

```
type Task {
  _id: ID!
  student: User!
  title: String!
  description: String
  dueDate: String!
  type: TaskType!
  priority: TaskPriority!
  completed: Boolean!
  completedAt: String
  subject: Subject
  createdAt: String!
}
```

```
type Review {
  _id: ID!
  professor: Professor!
  subject: Subject!
  student: User! # null si isAnonymous:true
  rating: Int!
  comment: String!
  isAnonymous: Boolean!
  helpfulCount: Int!
  hasVotedHelpful: Boolean! # resolver: verifica si el usuario actual votó
  isVisible: Boolean!
  createdAt: String!
}
```

```
type Resource {
  _id: ID!
  subject: Subject!
  student: User!
  title: String!
  url: String!
  type: ResourceType!
  description: String
  likeCount: Int!
  hasLiked: Boolean! # resolver: verifica si el usuario actual dio like
  isVisible: Boolean!
  createdAt: String!
}
```

```
type AuthPayload {
  token: String!
  user: User!
}
```

5.3 Queries completas

```

type Query {
  # Auth
  me: User          # usuario logueado desde el contexto JWT

  # Carreras
  careers: [Career!]!    # todas las carreras activas
  career(id: ID!): Career # una carrera por ID

  # Materias
  subjects(careerId: ID!, year: Int, semester: Int): [Subject!]!
  subject(id: ID!): Subject
  subjectsByCode(code: String!): Subject

  # Progreso personal del estudiante logueado
  myEnrollments(status: EnrollmentStatus): [Enrollment!]!
  myProgress: ProgressSummary!

  # Cuatrimestre
  currentSemester: CurrentSemester # trae el que tiene isActive:true
  semesters: [CurrentSemester!]!   # todos (solo admin)

  # Profesores
  professors(subjectId: ID, search: String): [Professor!]!
  professor(id: ID!): Professor

  # Tareas (solo las del usuario logueado)
  myTasks(completed: Boolean, type: TaskType): [Task!]!
  upcomingTasks(days: Int): [Task!]! # tareas de los próximos N días

  # Reseñas
  reviews(professorId: ID, subjectId: ID): [Review!]!

  # Recursos
  resources(subjectId: ID!, type: ResourceType): [Resource!]!

  # Admin
  users(careerId: ID, role: UserRole): [User!]! # solo admin
}

```

5.4 Mutations completas

```

type Mutation {
  # Auth
  register(input: RegisterInput!): AuthPayload!
  login(email: String!, password: String!): AuthPayload!
  updateProfile(name: String, avatar: String): User!
  changePassword(currentPassword: String!, newPassword: String!): Boolean!

  # Enrollments (progreso del alumno)
  upsertEnrollment(input: UpsertEnrollmentInput!): Enrollment!
  deleteEnrollment(id: ID!): Boolean!
}

```

```

# Tareas
createTask(input: CreateTaskInput!): Task!
updateTask(id: ID!, input: UpdateTaskInput!): Task!
completeTask(id: ID!): Task!
deleteTask(id: ID!): Boolean!

# Reseñas
createReview(input: CreateReviewInput!): Review!
voteHelpful(reviewId: ID!): Review!
deleteReview(id: ID!): Boolean!

# Recursos
createResource(input: CreateResourceInput!): Resource!
likeResource(resourceId: ID!): Resource!
deleteResource(id: ID!): Boolean!

# Admin - Carreras
createCareer(input: CareerInput!): Career!
updateCareer(id: ID!, input: CareerInput!): Career!
toggleCareerActive(id: ID!): Career!

# Admin - Materias
createSubject(input: SubjectInput!): Subject!
updateSubject(id: ID!, input: SubjectInput!): Subject!
toggleSubjectActive(id: ID!): Subject!

# Admin - Profesores
createProfessor(input: ProfessorInput!): Professor!
updateProfessor(id: ID!, input: ProfessorInput!): Professor!

# Admin - Cuatrimestre
createSemester(input: SemesterInput!): CurrentSemester!
activateSemester(id: ID!): CurrentSemester!
addOffering(semesterId: ID!, input: OfferingInput!): CurrentSemester!
removeOffering(semesterId: ID!, subjectId: ID!): CurrentSemester!

# Admin - Usuarios
createUser(input: RegisterInput!): User!
toggleUserActive(id: ID!): User!
}

```

5.5 Input types

```

input RegisterInput {
  name: String!
  email: String!
  password: String!
  legajo: String!
  careerId: ID!
  startYear: Int!
  role: UserRole
}

```

```
input UpsertEnrollmentInput {  
  subjectId: ID!  
  status: EnrollmentStatus!  
  grade: Float  
  approvedDate: String  
  enrolledSemester: String!  
  attempts: Int  
  notes: String  
}
```

```
input CreateTaskInput {  
  title: String!  
  description: String  
  dueDate: String!  
  type: TaskType!  
  priority: TaskPriority  
  subjectId: ID  
}
```

```
input CreateReviewInput {  
  professorId: ID!  
  subjectId: ID!  
  rating: Int!  
  comment: String!  
  isAnonymous: Boolean  
}
```

```
input CreateResourceInput {  
  subjectId: ID!  
  title: String!  
  url: String!  
  type: ResourceType!  
  description: String  
}
```

6. Resolvers y lógica de negocio

6.1 Contexto de Apollo Server

El contexto se construye en cada request extrayendo el token JWT del header Authorization. Si el token es válido, se decodifica y se agrega el usuario (sin password) al contexto. Los resolvers acceden a context.user para verificar autenticación y rol.

```
// server/src/index.js
const { ApolloServer } = require('@apollo/server');
const { expressMiddleware } = require('@apollo/server/express4');
const { verifyToken } = require('./middleware/auth');

const server = new ApolloServer({ typeDefs, resolvers });
await server.start();

app.use('/graphql', expressMiddleware(server, {
  context: async ({ req }) => {
    const token = req.headers.authorization?.replace('Bearer ', '');
    const user = token ? verifyToken(token) : null;
    return { user, dataloaders: createDataLoaders() };
  }
}));
```

6.2 Helpers de autenticación en resolvers

```
// Helpers reutilizables en cada resolver
const requireAuth = (context) => {
  if (!context.user) throw new GraphQLError('No autenticado', {
    extensions: { code: 'UNAUTHENTICATED' }
  });
  return context.user;
};

const requireAdmin = (context) => {
  const user = requireAuth(context);
  if (user.role !== 'admin') throw new GraphQLError('Sin permisos', {
    extensions: { code: 'FORBIDDEN' }
  });
  return user;
};
```

6.3 Resolver de ProgressSummary

Este es el cálculo más importante del sistema. Se ejecuta cuando se pide el campo progress de un User. Recibe el usuario del campo padre y calcula todos los indicadores sobre sus enrollments.

```
User: {
  progress: async (parent) => {
```

```

const career = await Career.findById(parent.career);
const totalSubjects = await Subject.countDocuments({
  career: parent.career, isActive: true, isElective: false
});
const enrollments = await Enrollment.find({ student: parent._id })
  .populate('subject');

const passed = enrollments.filter(e => e.status === 'passed');
const failed = enrollments.filter(e => e.status === 'failed');
const enrolled = enrollments.filter(e => e.status === 'enrolled');
const free = enrollments.filter(e => e.status === 'free');
const pending = totalSubjects - passed.length - failed.length
  - enrolled.length - free.length;

const grades = passed.filter(e => e.grade !== null).map(e => e.grade);
const average = grades.length
  ? grades.reduce((a, b) => a + b, 0) / grades.length : 0;

// Promedio ponderado por créditos
let totalCredits = 0, weightedSum = 0;
passed.forEach(e => {
  const credits = e.subject?.credits || 1;
  if (e.grade) { weightedSum += e.grade * credits; totalCredits += credits; }
});
const weightedAverage = totalCredits ? weightedSum / totalCredits : 0;

return {
  totalSubjects, passed: passed.length, failed: failed.length,
  enrolled: enrolled.length, pending, free: free.length,
  percentage: Math.round((passed.length / totalSubjects) * 100),
  average: Math.round(average * 100) / 100,
  weightedAverage: Math.round(weightedAverage * 100) / 100
};
}
}

```

6.4 DataLoaders — evitar el problema N+1

GraphQL puede generar muchas queries individuales a MongoDB cuando se populan referencias (por ejemplo, cargar el professor de cada offering de un cuatrimestre). DataLoader agrupa todas las llamadas a la misma colección en una sola query batch, reduciendo el número de operaciones a la base de datos.

```

// server/src/utils/dataLoaders.js
const DataLoader = require('dataloader');

const createDataLoaders = () => ({
  subjectLoader: new DataLoader(async (ids) => {
    const subjects = await Subject.find({ _id: { $in: ids } });
    return ids.map(id => subjects.find(s => s._id.equals(id)));
  }),
  professorLoader: new DataLoader(async (ids) => {
    const profs = await Professor.find({ _id: { $in: ids } });

```

```
    return ids.map(id => profs.find(p => p._id.equals(id)));
  }},
  careerLoader: new DataLoader(async (ids) => {
    const careers = await Career.find({ _id: { $in: ids } });
    return ids.map(id => careers.find(c => c._id.equals(id)));
  }},
});
```


7. Frontend — Apollo Client y estructura de queries

7.1 Configuración de Apollo Client

```
// client/src/apollo/client.js
import { ApolloClient, InMemoryCache, createHttpLink } from '@apollo/client';
import { setContext } from '@apollo/client/link/context';

const httpLink = createHttpLink({
  uri: import.meta.env.VITE_GRAPHQL_URL || 'http://localhost:4000/graphql',
});

const authLink = setContext((_, { headers }) => {
  const token = sessionStorage.getItem('token'); // sessionStorage > localStorage
  return { headers: { ...headers, authorization: token ? `Bearer ${token}` : '' } };
});

export const client = new ApolloClient({
  link: authLink.concat(httpLink),
  cache: new InMemoryCache({
    typePolicies: {
      User: { keyFields: ['_id'] },
      Subject: { keyFields: ['_id'] },
      Enrollment: { keyFields: ['_id'] },
    }
  }),
  defaultOptions: {
    watchQuery: { fetchPolicy: 'cache-and-network' }
  }
});
```

7.2 Ejemplos de queries del cliente

```
// client/src/graphql/queries.js
import { gql } from '@apollo/client';

export const GET_MY_PROGRESS = gql`
  query GetMyProgress {
    me {
      _id name legajo
      career { _id name shortName durationYears }
      progress {
        totalSubjects passed failed enrolled pending free
        percentage average weightedAverage
      }
    }
  }
`;
```

```
export const GET_MY_SUBJECTS = gql`
  query GetMySubjects($careerId: ID!) {
    subjects(careerId: $careerId) {
      _id name code year semester area isElective
    }
  }
`;
```

```

    prerequisites { _id name code }
  }
  myEnrollments {
    _id status grade approvedDate enrolledSemester attempts
    subject { _id }
  }
}
`;

```

```

export const GET_CURRENT_SEMESTER = gql`
  query GetCurrentSemester {
    currentSemester {
      _id label code startDate endDate
      offerings {
        subject { _id name code year semester }
        professors { _id name avgRating reviewCount }
        schedule classroom
        career { _id shortName }
      }
    }
  }
`;

```

```

export const GET_MY_TASKS = gql`
  query GetMyTasks($completed: Boolean, $type: TaskType) {
    myTasks(completed: $completed, type: $type) {
      _id title description dueDate type priority completed completedAt
      subject { _id name code }
    }
  }
`;

```

```

export const GET_REVIEWS = gql`
  query GetReviews($professorId: ID, $subjectId: ID) {
    reviews(professorId: $professorId, subjectId: $subjectId) {
      _id rating comment isAnonymous helpfulCount hasVotedHelpful createdAt
      professor { _id name avgRating reviewCount }
      subject { _id name }
      student { _id name }
    }
  }
`;

```

7.3 Ejemplos de mutations del cliente

```

// client/src/graphql/mutations.js
export const LOGIN = gql`
  mutation Login($email: String!, $password: String!) {
    login(email: $email, password: $password) {
      token
      user { _id name email role legajo career { _id name shortName } }
    }
  }
`;

```

```
export const UPSERT_ENROLLMENT = gql`
  mutation UpsertEnrollment($input: UpsertEnrollmentInput!) {
    upsertEnrollment(input: $input) {
      _id status grade approvedDate enrolledSemester attempts
      subject { _id name code }
    }
  }
`;
```

```
export const CREATE_TASK = gql`
  mutation CreateTask($input: CreateTaskInput!) {
    createTask(input: $input) {
      _id title dueDate type priority completed
      subject { _id name }
    }
  }
`;
```

```
export const VOTE_HELPFUL = gql`
  mutation VoteHelpful($reviewId: ID!) {
    voteHelpful(reviewId: $reviewId) {
      _id helpfulCount hasVotedHelpful
    }
  }
`;
```

8. Rutas del frontend y pantallas del MVP

8.1 Tabla de rutas

Ruta	Página	Acceso y query principal
/login	LoginPage	Pública. Mutation: login. Redirige a /dashboard si ya hay sesión.
/register	LoginPage (tab)	Pública. Mutation: register. Requiere elegir carrera (query: careers).
/dashboard	DashboardPage	Privado. Query: me { progress, enrollments cursando }. Tareas urgentes.
/materias	SubjectsPage	Privado. Query: subjects + myEnrollments. Tabla con estado por materia.
/cuatrimestre	SemesterPage	Privado. Query: currentSemester. Cards por materia con profesores.
/calendario	CalendarPage	Privado. Query: myTasks. Mutation: createTask, completeTask, deleteTask.
/recomendaciones	ReviewsPage	Privado. Query: professors, reviews. Mutation: createReview, voteHelpful.
/recursos	ResourcesPage	Privado. Query: resources. Mutation: createResource, likeResource.
/calculadora	CalculatorPage	Privado. Sin query propia: usa los datos de myEnrollments del caché.
/perfil	ProfilePage	Privado. Query: me. Mutation: updateProfile, changePassword.
/admin	AdminPage	Solo admin. Queries y mutations de carreras, materias, profesores, usuarios.

8.2 Inventario de pantallas del MVP

#	Pantalla	Contenido principal	Acciones
M-01	Login / Registro	Tabs: ingresar y crear cuenta. En registro: selector de carrera con descripción.	login, register mutations.
M-02	Dashboard	4 stat cards (% aprobadas, cursando, promedio). Barra de progreso. Próximas tareas. Accesos rápidos.	Navegar módulos. Completar tarea directamente.
M-03	Plan de estudios	Grid de materias por año. Estado coloreado (aprobada/cursando/pendiente/desaprobada). Nota si corresponde.	Click en materia → modal para cambiar estado y cargar nota.

#	Pantalla	Contenido principal	Acciones
M-04	Cuatrimestre en curso	Cards de materias activas, filtradas por la carrera del alumno. Nombre y rating de cada profesor.	Click en profesor → ver reseñas.
M-05	Calendario de tareas	Lista de tareas ordenadas por fecha. Indicadores de urgencia. Filtros por tipo.	Crear, completar, eliminar tareas.
M-06	Modal nueva tarea	Formulario: título, fecha, tipo, prioridad, materia relacionada.	createTask mutation. Cancelar.
M-07	Recomendaciones	Lista de profesores con avgRating. Reseñas con comentarios. Filtro por materia.	createReview, voteHelpful mutations.
M-08	Recursos	Recursos por materia ordenados por likeCount. Filtro por tipo.	createResource, likeResource mutations.
M-09	Perfil	Datos del alumno. Stats de progreso. Editar nombre.	updateProfile, changePassword, logout.
M-10	Admin panel	Tabs: Carreras, Materias, Profesores, Cuatrimestre, Usuarios. Tablas con ABM.	Todas las mutations admin.

9. Configuración y comandos

9.1 Variables de entorno — server/.env

```
MONGO_URI=mongodb://localhost:27017/infotrack
JWT_SECRET=infotrack_super_secret_minimo_32_chars
JWT_EXPIRES_IN=8h
PORT=4000
NODE_ENV=development
# FRONTEND_URL=http://localhost:5173 # para configurar CORS
```

9.2 Variables de entorno — client/.env

```
VITE_GRAPHQL_URL=http://localhost:4000/graphql
```

9.3 Instalación del backend

```
npm init -y
npm install @apollo/server graphql express cors dotenv
npm install mongoose bcryptjs jsonwebtoken dataloader
npm install -D nodemon
```

9.4 Instalación del frontend

```
npm create vite@latest client -- --template react
npm install @apollo/client graphql react-router-dom recharts
```

9.5 Scripts package.json del backend

```
"scripts": {
  "dev": "nodemon src/index.js",
  "start": "node src/index.js",
  "seed": "node src/utils/seed.js"
}
```

10. Alcance del MVP y plan de desarrollo

10.1 Módulos del MVP vs. versión completa

Módulo	En el MVP	En versiones futuras
Auth	Registro con carrera, login JWT, rutas protegidas por rol.	Recupero de contraseña, OAuth universitario.
Dashboard	Stat cards, barra de progreso, tareas urgentes.	Gráfico histórico del promedio, mapa visual.
Plan de estudios	Estado y nota por materia. Agrupado por año.	Mapa de correlativas, simulador de cursada.
Cuatrimestre	Materias activas + profesores + horario.	Alertas de inscripción, integración con SIU.
Calendario	CRUD de tareas con fecha, tipo y prioridad.	Vista mensual, notificaciones push o email.
Recomendaciones	Reseñas con rating, comentario y voto útil.	Ranking con filtros avanzados, moderación.
Recursos	Compartir links con tipo + dar like.	Subida de archivos (Cloudinary), moderación.
Calculadora	Promedio simple y ponderado por créditos.	Simulación de notas futuras, reglas por carrera.
Admin panel	ABM completo de todas las entidades base.	Reportes de uso, estadísticas globales.

10.2 Plan de desarrollo — 4 semanas

Semana	Backend (Node.js + Apollo + MongoDB)	Frontend (React + Apollo Client)
1	Setup proyecto. Modelos Career, Subject, User. Schema GraphQL: tipos Career, Subject, User, AuthPayload. Resolvers de auth (register, login, me). Contexto JWT en Apollo Server.	Setup Vite + React. Configuración de Apollo Client con authLink. AuthContext. LoginPage con tabs. PrivateRoute y AdminRoute. Conexión real con mutation login/register.
2	Modelo Enrollment. Schema: tipos Enrollment, ProgressSummary. Resolvers myEnrollments, myProgress, upsertEnrollment. DataLoaders para Subject y Career. Modelo CurrentSemester con subdocumento SemesterOffering.	DashboardPage con query me { progress } y stat cards. SubjectsPage con tabla de materias por año y modal para cambiar estado/nota. SemesterPage con cards por materia y profesores.
3	Modelo Professor, Task, Review, Resource. Schema completo. Resolvers de tasks (myTasks, CRUD), reviews (con voteHelpful y recálculo en Professor),	CalendarPage con lista de tareas y modal nueva tarea. ReviewsPage con lista de profesores y reseñas. ResourcesPage con

Semana	Backend (Node.js + Apollo + MongoDB)	Frontend (React + Apollo Client)
	resources (con likeResource). Resolvers admin completos.	filtros y likes. CalculatorPage con datos del caché de enrollments.
4	Script seed con datos realistas de las 4 carreras. Validaciones completas en inputs. Manejo de errores GraphQL (codes: UNAUTHENTICATED, FORBIDDEN, BAD_USER_INPUT). Pruebas en Apollo Sandbox.	ProfilePage. AdminPage con tabs y ABM completo. Manejo de estados de carga (loading/error). Pulido visual CSS. Testing manual de todos los flujos. Preparación para entrega.

11. Criterios de aceptación del MVP

11.1 Autenticación

1. La mutation register crea un usuario y retorna un token JWT válido.
2. La mutation login retorna token + user. Las credenciales incorrectas devuelven error UNAUTHENTICATED.
3. Las queries y mutations protegidas sin token retornan error UNAUTHENTICATED.
4. Las operaciones de admin sin rol admin retornan error FORBIDDEN.

11.2 Progreso académico

5. La query me { progress } retorna correctamente el % de avance, aprobadas, en curso y promedio.
6. La mutation upsertEnrollment permite registrar o actualizar el estado de una materia.
7. Al marcar una materia como aprobada con nota, el promedio en el dashboard se actualiza.
8. El porcentaje de avance es correcto: materias aprobadas / total de materias del plan.

11.3 Cuatrimestre y recomendaciones

9. La query currentSemester retorna solo las materias de la carrera del alumno logueado.
10. La mutation createReview guarda la reseña y actualiza avgRating y reviewCount en Professor.
11. La mutation voteHelpful no permite votar dos veces al mismo usuario (helpfulVoters evita duplicados).

11.4 Calendario y recursos

12. Se pueden crear, completar y eliminar tareas. Las completadas aparecen separadas de las pendientes.
13. La query upcomingTasks retorna solo las tareas de los próximos N días sin completar.
14. La mutation likeResource no permite dar like dos veces al mismo recurso.

12. Escalabilidad — Roadmap v2 y v3

12.1 Por qué GraphQL facilita la escalabilidad

- Agregar un nuevo tipo y sus resolvers no modifica ninguna query existente del cliente.
- El caché de Apollo Client se actualiza automáticamente con las respuestas de mutations (usando `cache.modify` o `refetchQueries`).
- Las subscriptions GraphQL permiten agregar tiempo real (notificaciones, alertas) sin cambiar la arquitectura.
- El schema sirve como contrato: el equipo de frontend puede trabajar con mocks mientras el backend implementa los resolvers.

12.2 Funcionalidades de versión 2

- Mapa visual de correlativas: tipo React Flow o SVG, con colores por estado de cada materia.
- Simulador de cursada: marcar materias futuras y ver el impacto en progreso sin guardar datos reales.
- Alertas de regularidad: campo `regularityExpires` en `Enrollment` comparado con la fecha actual.
- Estadísticas personales: gráfico de evolución del promedio por cuatrimestre usando `Recharts LineChart`.
- GraphQL Subscriptions: alertas en tiempo real cuando el admin activa un nuevo cuatrimestre.

12.3 Funcionalidades de versión 3

- Integración con SIU Guaraní: importar materias aprobadas automáticamente si hay API disponible.
- App móvil: React Native consumiendo la misma API GraphQL sin cambios en el backend.
- Moderación avanzada: reportes de reseñas y recursos, queue de moderación para el admin.
- Notificaciones por email: `Nodemailer/SendGrid` triggered desde resolvers para tareas próximas a vencer.
- Dashboard global del admin: estadísticas agregadas con MongoDB pipeline `$group + $lookup`.